

Анализ приложения SupportDesk

Это десктопное WinForms-приложение (.NET Framework 4.8) для службы технической поддержки / helpdesk. Основные возможности:

- Роли: **ADMIN, WORKER, USER**
- Заявки (tickets), инвентарь, лицензии, картриджи/принтеры, контакты, пароли, справочники
- SQLite-база (`teh_db.db`)
- Кастомный UI (собственные контролы `ExpUIButton` , `textBoxUI` , `ExpUICheckBox` и т.д. в стиле Material)
- Telegram-бот, email-уведомления, экспорт в Excel, «запомнить меня», календарь, аватарки

Запуск: `PreLoaderForm` → `LoginForm` → одна из главных форм (`MainAdminForm` / `MainWorkerForm` / `MainUserForm`).

1. Критические проблемы безопасности (влияют и на UX)

Проблема	Где	Последствия
Хардкод паролей почты	<code>Email.mailPassword</code> , <code>yandexPassword</code>	Любой может прочитать исходники / декомпилировать и получить доступ к почте
Хардкод Telegram-токена	<code>TelegramConfig.botToken</code>	Полный контроль над ботом
Root-пароль в коде	<code>RootAccess.rootPassword</code> = " <code>K00pt3hR00t4cce\$\$</code> "	Обход защиты
Пароли в plain-text в БД + файл «запомнить меня»	<code>loginPassword.txt</code> в <code>%TEMP%\SupportDesk\</code>	Легко украсть
Список слабых паролей только клиентский	<code>CertificateSecurity.noPasswords</code>	Можно обойти

Это не только security, но и UX: при утечке пользователь теряет доверие, а восстановление пароля через email становится риском.

2. Ошибки логики навигации и жизненного цикла форм (самое серьёзное для UX)

Паттерн почти везде одинаковый:

C#

```
Form fMain = new MainAdminForm();  
fMain.Show();  
fMain.FormClosed += Form_FormClosed;  
Hide();
```

Проблемы:

1. Каждый переход создаёт новый экземпляр главной формы

- Теряется состояние фильтров, выбранных строк, позиции скrolла, открытых виджетов.
- Растёт потребление памяти (особенно при частом переключении Админ ↔ Картриджи ↔ Пользователи и т.д.).

2. Цепочка `FormClosed` → `Close()`

При закрытии любой дочерней формы закрывается вся цепочка. Пользователь часто «вылетает» из приложения неожиданно.

3. `DataRequests.IsOpen` как глобальный флаг

Используется как примитивная блокировка открытия нескольких одинаковых окон. Работает плохо: флаги не всегда сбрасываются, возможны «залипшие» состояния, когда окно нельзя открыть повторно.

4. `TelegramForm` всегда показывается при логине (`fbot.Show()` в `LoginForm_Load`). Даже если пользователю он не нужен — лишнее окно.

5. Выход из дочерних форм часто делает `new MainXxxForm()` вместо возврата к уже существующей.

Это классическая анти-паттерн-навигация WinForms.

Правильный подход: один экземпляр главной формы + `UserControls` / `TabControl` / `Panel.Show/Hide` или `MDI`, либо как минимум кэширование форм.

3. UI/UX-проблемы интерфейса

3.1. Фиксированные размеры и отсутствие адаптивности

- Почти все формы имеют жёстко прописанные Size / Location (200×35 кнопки в сайдбаре, 200px ширина меню и т.д.).
- WidgetsConfig содержит магические числа:

C#

```
sizeGroupBoxOnWidgets = 503;  
locationGroupBoxOffWidgets = 38;  
locationFirstWidget = 212;
```

- Нет Anchor / Dock в ключевых местах, нет обработки Resize . На экранах с другим DPI / разрешением / масштабированием Windows интерфейс «ломается».

3.2. Кастомные контролы

- ExpUIButton , textBoxUI и т.д. — хорошая идея, но:
 - Нет поддержки TabIndex / клавиатурной навигации в полной мере.
 - Нет accessibility (Screen Reader, high-contrast).
 - Hover/Click-эффекты реализованы вручную, возможны артефакты при быстром движении мыши.

3.3. MessageBox

C#

```
if (!DataUsers.UseSystemNotifaction)  
{  
    return;    // ← уведомление просто не показывается!  
}
```

Инвертированная логика + опечатка в названии свойства (Notifaction).
Пользователь может отключить уведомления и вообще не видеть ошибки.

3.4. Валидация форм

- В `CreateNewTicketForm` кнопка активируется только если все поля заполнены (включая 4 поля времени).
Это жёстко: пользователь не может сохранить черновик.
- Много мест с проверкой `== ""` вместо `string.IsNullOrEmpty`.
- Ошибки часто показываются через кастомный `MessageBox` с текстом `ex.Message / ex.Source` — пользователю это ничего не говорит.

3.5. PreLoader

- Создаёт папки и файлы через `Task`, но с `Task.Delay(1000)` внутри — искусственная задержка.
- `CreateAndUpdateFileRemember` запускается без `.Wait()` — возможна гонка при первом запуске.

3.6. Роли и права

- После 3 неудачных попыток аккаунт блокируется (`typeAccess = "LOCKED"`).
Хорошо, но нет понятного UI для администратора «разблокировать пользователя» в удобном месте (нужно лезть в справочник пользователей).
- `WORKER` и `USER` имеют сильно урезанный функционал, но визуально интерфейс почти такой же — пользователь не всегда понимает, почему кнопка «не работает».

4. Логические и архитектурные ошибки

Проблема	Описание
Глобальное состояние	<code>DataUsers</code> , <code>DataRequests</code> , <code>DataSpecificUser</code> — всё <code>static</code> . Легко получить рассинхрон при параллельных операциях.
<code>async void</code> повсеместно	100 + мест. Исключения в таких методах «проглатываются» или роняют приложение.
SQLite без транзакций в сложных операциях	При ошибке посередине данные могут остаться в неконсистентном состоянии.
Нет пагинации	<code>SELECT * FROM Requests</code> — при большом количестве заявок <code>DataGridView</code> будет тормозить.
Хардкод лимитов	<code>limitUsers = 500</code> , <code>limitTech = 5000</code> — защиты в код.
Логирование	Просто <code>File.AppendAllText</code> без ротации и без уровней.

5. Положительные моменты

- Есть разделение ролей.
 - Кастомный UI выглядит современнее стандартного WinForms.
 - Есть «глазик» для пароля, «запомнить меня», проверка слабых паролей.
 - Асинхронная загрузка данных из БД (хотя и через `async void`).
 - Попытка сделать единый стиль кнопок и `MessageBox`.
-

Рекомендации по приоритету

Высокий приоритет (сразу):

1. Убрать все секреты из кода (токены, пароли почты, root-пароль) → конфигурация / DPAPI / Azure Key Vault и т.п.
2. Переделать навигацию: один экземпляр главной формы + `UserControl`'ы или правильный `Show/Hide`.
3. Исправить `MessageBoxUI` (логика `UseSystemNotification`).
4. Заменить глобальные `static`-классы данных на нормальный сервис / `ViewModel`.

Средний приоритет:

5. Сделать формы адаптивными (`Anchor/Dock` + минимум `Resize`-логики).
6. Добавить нормальную валидацию с понятными сообщениями и возможностью черновиков.
7. Убрать искусственные `Task.Delay` в `PreLoader`.
8. Добавить пагинацию / виртуализацию в гридах.

Низкий приоритет (улучшения UX):

9. Клавиатурная навигация и `hotkeys`.
 10. Тёмная тема / поддержка системного масштабирования.
 11. Онбординг / подсказки для новых пользователей.
 12. История действий и более информативные логи.
-

Если нужно, могу сделать более глубокий разбор конкретной формы (например, `MainAdminForm` или `CreateNewTicketForm`), предложить рефакторинг навигации или набросать схему правильной архитектуры.